TEMA 2: ADMINISTRACIÓN AVANZADA, DESPLIEGUE SEGURO Y MONITORIZACIÓN DE SERVIDORES WEB

Unidad Formativa para el Resultado de Aprendizaje RA2 (Criterios a - j)
Guía Técnica y Prácticas de Aula

1. Introducción al RA2 y Marco de Competencias

El Resultado de Aprendizaje 2 (RA2) del módulo de Despliegue de Aplicaciones Web establece las competencias necesarias para implantar aplicaciones web en servidores web de producción, evaluando y aplicando criterios de configuración para su funcionamiento seguro y de alto rendimiento. En el Tema 1 se abordó la instalación básica de servidores HTTP (Apache), entornos de ejecución de aplicaciones dinámicas (Python/Flask) y virtualización/contenedores (Docker en AWS EC2). En este Tema 2 se da el salto definitivo hacia la administración avanzada, el bastionado (*hardening*), las comunicaciones cifradas mediante HTTPS y la monitorización en tiempo real.

2. Administración Avanzada y Módulos del Servidor Web (RA2.a, b, c, h)

2.1. Parámetros de Administración y Modelos MPM (Multi-Processing Modules)

La administración profesional de un servidor web exige optimizar la concurrencia y el uso de memoria según la tipología del tráfico entrante. Apache gestiona las peticiones mediante los módulos MPM (*Multi-Processing Modules*):

- • MPM Prefork: Crea procesos hijo independientes para cada petición. No utiliza hilos. Es el modelo más estable y compatible con librerías no seguras para hilos (thread-unsafe) como PHP tradicional, aunque consume mayor cantidad de memoria RAM.
- • MPM Worker: Implementa una arquitectura híbrida donde cada proceso hijo crea múltiples hilos (threads) de ejecución. Permite atender un número significativamente mayor de clientes simultáneos con menor huella de memoria.
- • MPM Event: Basado en MPM Worker, pero optimizado para peticiones Keep-Alive. Delega el mantenimiento de conexiones inactivas a un hilo dedicado, liberando los hilos de trabajo para peticiones activas. Es el modelo recomendado en producción moderna.

> **Directivas Clave de Rendimiento en Apache (httpd.conf / apache2.conf)**
> • Timeout 60: Tiempo máximo en segundos que el servidor espera para recibir una petición o enviar una respuesta.
> • KeepAlive On: Permite mantener conexiones HTTP頭 persistentes para múltiples peticiones por cliente.
> • MaxKeepAliveRequests 100: Número máximo de peticiones permitidas por conexión persistente.
> • KeepAliveTimeout 5: Segundos que el servidor espera una nueva petición antes de cerrar la conexión Keep-Alive.

2.2. Ampliación de Funcionalidad mediante Módulos (`a2enmod` / `a2dismod`)

En distribuciones Debian/Ubuntu/MAX, la gestión de módulos se realiza mediante los comandos de administración `a2enmod` (activar) y `a2dismod` (desactivar), los cuales crean o eliminan enlaces simbólicos entre `/etc/apache2/mods-available/` y `/etc/apache2/mods-enabled/`.

Los módulos imprescindibles en entornos de producción incluyen:

- **mod_rewrite:** mod_rewrite: Motor de reescritura de URLs mediante reglas basadas en expresiones regulares. Fundamental para frameworks MVC, URLs amigables y redirecciones HTTPS.
- **mod_headers:** mod_headers: Permite modificar y añadir cabeceras de respuesta HTTP para implementar políticas de seguridad (HSTS, CSP, X-Frame-Options).
- **mod_deflate:** mod_deflate: Comprime dinámicamente el contenido de texto (HTML, CSS, JS, JSON) utilizando el algoritmo Gzip antes de enviarlo al navegador, reduciendo el ancho de banda consumido hasta en un 70%.
- **mod_proxy:** mod_proxy / mod_proxy_http: Habilita a Apache para actuar como Proxy Inverso, reexpidiendo peticiones entrantes hacia servidores de aplicaciones internos (Node.js, Tomcat, Flask, Docker).
- **mod_security:** mod_security: Cortafuegos de aplicación web (WAF) que inspecciona el tráfico HTTP/HTTPS en busca de patrones de ataque conocidos (Inyección SQL, XSS, Path Traversal).

```
# Activación de módulos esenciales y reinicio del servicio
sudo a2enmod rewrite headers deflate proxy proxy_http ssl
sudo systemctl restart apache2
```

2.3. Configuración de Sitios Virtuales (VirtualHosts) y Jerarquía de Directivas

Los sitios virtuales (`VirtualHost`) permiten a un único servidor web albergar múltiples dominios o aplicaciones independientes compartiendo la misma dirección IP y puerto. La configuración de un VirtualHost combina directivas de ámbito:

```
<VirtualHost *:80>
    ServerName portal.empresa.local
    ServerAlias www.portal.empresa.local
    ServerAdmin admin@empresa.local

    DocumentRoot /var/www/portal/html

    # Directiva de ámbito de directorio
    <Directory /var/www/portal/html>
        Options -Indexes +FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/portal_error.log
    CustomLog ${APACHE_LOG_DIR}/portal_access.log combined
</VirtualHost>
```

El orden de precedencia y jerarquía en el procesamiento de directivas de acceso es el siguiente: `<Directory>` (procesado en orden de ruta más corta a más larga) → `<Location>`

(procesado según la URL solicitada) → `.htaccess` (procesado dinámicamente por petición si `AllowOverride` está activado).

3. Autenticación, Control de Acceso y Bastionado / Hardening (RA2.d, g)

3.1. Control de Acceso por IP Origen y Redes

En Apache 2.4, el control de acceso se gestiona mediante el módulo `mod_authz_core` y la directiva `Require`:

```
# Permitir acceso solo a la red local corporativa
<Directory /var/www/portal/admin>
    Require ip 192.168.1.0/24
    Require ip 10.0.0.15
</Directory>

# Denegar el acceso a archivos de configuración sensibles
<FilesMatch "^\.(ht|git|env)">
    Require all denied
</FilesMatch>
```

3.2. Mecanismos de Autenticación HTTP (Basic y Digest)

Cuando se requiere proteger un área del servidor mediante contraseña a nivel de infraestructura web (sin pasar por la aplicación), se utiliza la autenticación HTTP Basic o Digest.

Paso 1: Creación del archivo de contraseñas de forma segura utilizando `htpasswd` (con cifrado bcrypt o MD5):

```
# Creación del archivo por primera vez (-c) con contraseña cifrada
sudo htpasswd -c /etc/apache2/.htpasswd usuario_admin
```

Paso 2: Configuración de la directiva de protección en el VirtualHost o `.htaccess`:

```
<Directory /var/www/portal/privado>
    AuthType Basic
    AuthName "Área Restringida de Administración"
    AuthUserFile /etc/apache2/.htpasswd
    Require valid-user
</Directory>
```

3.3. Bastionado (Hardening) del Servidor Web

El bastionado u *ocultamiento de firmas* evita proporcionar información valiosa a los escáneres de vulnerabilidades automatizados. Se deben modificar los siguientes parámetros globales en `/etc/apache2/conf-available/security.conf`:

Directiva	Valor por Defecto	Valor de Bastionado (Recomendado)	Efecto de Seguridad
ServerTokens	Full	Prod	Oculta la versión exacta del S.O. y módulos. Devuelve solo 'Apache'.

ServerSignature	On	Off	Elimina el pie de página generado en errores HTTP (404, 500) con datos del servidor.
Options	Indexes FollowSymLinks	-Indexes	Evita que se muestre el listado de archivos cuando no existe index.html.
TraceEnable	On	Off	Deshabilita el método HTTP TRACE, previniendo ataques de Cross-Site Tracing (XST).

4. Comunicaciones Seguras: SSL/TLS y Certificados Digitales (RA2.e, f)

4.1. Criptografía Asimétrica, PKI y Tipos de Certificados

La seguridad en las comunicaciones cliente-servidor se apoya en una Infraestructura de Clave Pública (PKI). El cliente verifica la identidad del servidor mediante un Certificado Digital X.509 emmitido por una Autoridad de Certificación (CA) de confianza.

- **1.** Certificados Autofirmados: Generados localmente con herramientas como OpenSSL. No son reconocidos por los navegadores (muestran advertencia de seguridad), pero son ideales para entornos de desarrollo y pruebas internas.
- **2.** Certificados de Producción emmitidos por CA: Validados por entidades reconocidas (Let's Encrypt, DigiCert). Garantizan la autenticidad del dominio y el cifrado sin advertencias.

4.2. Generación de Claves y Certificados Autofirmados con OpenSSL

Para crear un certificado X.509 autofirmado de 2048 bits válido por 365 días en una sola línea de comandos:

```
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout /etc/ssl/private/apache-selfsigned.key \
  -out /etc/ssl/certs/apache-selfsigned.crt \
  -subj "/CN=servidor.local/O=Empresa/C=ES"
```

4.3. Automatización de Certificados Gratuitos con Let's Encrypt y Certbot

Let's Encrypt proporciona certificados SSL/TLS gratuitos e integrados mediante el protocolo ACME (*Automated Certificate Management Environment*). Certbot es la herramienta cliente que automatiza la validación y la renovación.

El proceso de validación del dominio por Certbot puede realizarse mediante dos mecanismos principales:

- • Desafío HTTP-01: Certbot coloca un archivo temporal en la ruta `/var/www/html/.well-known/acme-challenge/` y los servidores de Let's Encrypt realizan una petición HTTP por el puerto 80 para verificar el control del dominio.
- • Desafío DNS-01: Requiere crear un registro TXT específico en las zonas DNS del dominio. Es obligatorio cuando se desean obtener certificados comodín (*Wildcard*, ej. `*.empresa.com`).

```
# Instalación de Certbot y el conector para Apache
sudo apt update && sudo apt install -y certbot python3-certbot-apache
```

```
# Obtención e instalación automática del certificado para un dominio público
sudo certbot --apache -d mi-dominio.com -d www.mi-dominio.com
```

4.4. Configuración de HTTPS en Apache y Redirección HTTP -> HTTPS

Un VirtualHost seguro en el puerto 443 requiere habilitar el módulo `mod_ssl` y especificar la ubicación de la clave privada y la cadena de certificados:

```
<VirtualHost *:443>
    ServerName mi-dominio.com
    DocumentRoot /var/www/html

    SSLEngine on
    SSLCertificateFile /etc/letsencrypt/live/mi-dominio.com/fullchain.pem
    SSLCertificateKeyFile /etc/letsencrypt/live/mi-dominio.com/privkey.pem

    # Robustecimiento de cifrado (SSL/TLS Protocols)
    SSLProtocol all -SSLv3 -TLSv1 -TLSv1.1 +TLSv1.2 +TLSv1.3
    SSLCipherSuite HIGH:!aNULL:!MD5:!3DES
</VirtualHost>

# Redirección obligatoria de HTTP (Puerto 80) a HTTPS (Puerto 443)
<VirtualHost *:80>
    ServerName mi-dominio.com
    Redirect permanent / https://mi-dominio.com/
</VirtualHost>
```

> 📌 **Cabeceras de Seguridad Avanzada (HTTP Response Headers)**
> • Strict-Transport-Security (HSTS): Obliga al navegador a comunicarse exclusivamente mediante HTTPS durante el tiempo especificado (ej. max-age=31536000).
> • X-Frame-Options SAMEORIGIN: Previene ataques de Clickjacking impidiendo que la web sea incrustada en iframes externos.
> • X-Content-Type-Options nosniff: Evita que el navegador interprete archivos con tipos MIME incorrectos o modificados.
> • Content-Security-Policy (CSP): Restringe los orígenes desde los cuales se pueden cargar scripts, estilos e imágenes.

5. Despliegue Seguro en la Nube y Contenedores (RA2.i)

5.1. Arquitectura de Proxy Inverso con SSL Termination en Contenedores

En la arquitectura de microservicios moderna, los contenedores de aplicación (Node.js, Flask, Django) ejecutan servidores web ligeros en puertos internos. Se sitúa un servidor web Nginx o Apache frontal como **Proxy Inverso con Terminación SSL (SSL Termination)**, el cual descifra el tráfico HTTPS y lo redirige por la red interna de Docker hacia el contenedor correspondiente.

```
# Ejemplo de docker-compose.yml para arquitectura Proxy Inverso + Web + Certbot
version: '3.8'

services:
  webserver:
    image: nginx:alpine
    ports:
```

```
    - "80:80"
    - "443:443"
  volumes:
    - ./nginx.conf:/etc/nginx/conf.d/default.conf
    - ./certbot/conf:/etc/letsencrypt
    - ./certbot/www:/var/www/certbot
  depends_on:
    - app_backend

app_backend:
  image: mi-app-flask:latest
  expose:
    - "5000"

certbot:
  image: certbot/certbot
  volumes:
    - ./certbot/conf:/etc/letsencrypt
    - ./certbot/www:/var/www/certbot
```

5.2. Despliegue Seguro Automatizado en AWS EC2 mediante User Data

Al desplegar en nubes públicas como AWS EC2, la seguridad debe configurarse tanto en la capa de red virtual (**Grupos de Seguridad / Security Groups**) abriendo únicamente los puertos 22 (SSH restringido), 80 (HTTP) y 443 (HTTPS), como en el script de arranque **User Data** que automatiza la configuración.

6. Gestión de Logs, Monitorización y Análisis en Tiempo Real (RA2.j)

6.1. Anatomía y Formatos de Logs en Servidores Web

Los archivos de registro (*logs*) son la fuente principal de información para la auditoría de seguridad y el análisis de rendimiento. Apache gestiona dos tipos de logs principales:

- **1.** AccessLog (Registro de Accesos): Registra cada petición HTTP recibida. El formato estándar 'combined' almacena: IP cliente, fecha/hora, método HTTP, URI solicitada, código de estado (200, 404, 500), tamaño de respuesta, Referer y User-Agent.
- **2.** ErrorLog (Registro de Errores): Almacena advertencias, fallos de ejecución, errores de compilación de scripts y problemas de arranque del servidor. El nivel de detalle se ajusta con la directiva `LogLevel` (debug, info, warn, error, crit).

6.2. Rotación de Logs con `logrotate`

Para evitar que los archivos de registros agoten el espacio en disco, el servicio del sistema `logrotate` comprime y rota periódicamente los archivos ubicados en `/var/log/apache2/`.

6.3. Monitorización Interactiva en Tiempo Real con GoAccess

GoAccess es un analizador de logs en tiempo real para servidores web que funciona tanto en la consola de comandos como generando un panel de control interactivo en HTML accesible vía web.

```
# Instalación de GoAccess en Linux
sudo apt install -y goaccess

# Ejecución en consola analizando el AccessLog de Apache
goaccess /var/log/apache2/access.log --log-format=COMBINED
```

```
# Generación de un Dashboard HTML en tiempo real y actualización continua (Daemon)
goaccess /var/log/apache2/access.log -o /var/www/html/reporte.html --log-format=COMBINED --real-time-html
```

6.4. Prevención Activa de Intrusiones con Fail2ban

Fail2ban es un demonio de prevención de intrusiones que examina los archivos de logs (`access.log` y `error.log`) en busca de patrones sospechosos (múltiples intentos fallidos de login, escaneo de vulnerabilidades, ataques DoS) y añade reglas dinámicas en el cortafuegos `iptables` / `nftables` para bloquear la IP del atacante durante un tiempo determinado.

```
# Ejemplo de configuración de un 'Jail' para Apache en /etc/fail2ban/jail.local
[apache-auth]
enabled  = true
port     = http,https
filter   = apache-auth
logpath  = /var/log/apache2/*error.log
maxretry = 3
findtime = 600
bantime  = 3600
```

7. Cuaderno de Prácticas Propuestas del Tema 2

A continuación se presentan las 4 prácticas de laboratorio del Tema 2. Cada práctica está diseñada para consolidar de forma aplicada los criterios de evaluación del RA2.

7.1. Práctica 1: Hardening, Módulos Avanzados y Control de Acceso en Apache

- • Criterios evaluados: RA2.a, RA2.b, RA2.c, RA2.d, RA2.g, RA2.h.
- • Objetivo: Bastionar un servidor Apache en un entorno Linux local, configurar autenticación digest/basic y habilitar mod_rewrite y compresión.

Desarrollo de la Práctica 1: Tareas a realizar paso a paso:

- 1. Configurar un VirtualHost para el dominio local `seguro.empresa.local` apuntando al directorio `/var/www/seguro`.
- 2. Editar `/etc/apache2/conf-available/security.conf` para establecer `ServerTokens Prod`, `ServerSignature Off` y deshabilitar el método TRACE.
- 3. Crear una carpeta `/var/www/seguro/admin` y protegerla mediante contraseñas HTTP utilizando `htpasswd` almacenado en `/etc/apache2/.htpasswd_seguro`.
- 4. Activar los módulos `mod_rewrite`, `mod_headers` y `mod_deflate`. Configurar una regla en `.htaccess` que redirija las peticiones de `/viejo` a `/nuevo`.
- 5. Redactar un documento breve con las comprobaciones de seguridad realizadas (`curl -I` verificando las cabeceras).

7.2. Práctica 2: Cifrado HTTPS y Automatización con Let's Encrypt / OpenSSL

- • Criterios evaluados: RA2.e, RA2.f, RA2.g.
- • Objetivo: Generar un certificado de desarrollo con OpenSSL y posteriormente desplegar un dominio público seguro mediante Certbot y Let's Encrypt.

Desarrollo de la Práctica 2: Tareas a realizar paso a paso:

- 1. Generar un par de claves RSA de 2048 bits y un certificado autofirmado con OpenSSL para el dominio de pruebas.
- 2. Configurar el archivo VirtualHost en el puerto 443 especificando `SSLEngine on`, las rutas de los certificados y restringir los protocolos a TLSv1.2 y TLSv1.3.
- 3. Configurar la redirección automática de todo el tráfico HTTP (puerto 80) hacia HTTPS (puerto 443).
- 4. Utilizar Certbot en un entorno con dominio público (o laboratorio simulado) para solicitar e instalar un certificado válido mediante el desafío HTTP-01.
- 5. Verificar la calidad del cifrado mediante una prueba de cabeceras HSTS y `X-Frame-Options` agregadas con `mod_headers`.

7.3. Práctica 3: Proxy Inverso SSL Termination en Contenedores sobre AWS EC2

- • Criterios evaluados: RA2.f, RA2.h, RA2.i.
- • Objetivo: Desplegar mediante Docker Compose una arquitectura de dos capas (Proxy Inverso Nginx/Apache + Aplicación Backend Flask/Node) sobre una instancia AWS EC2 automatizada con User Data.

Desarrollo de la Práctica 3: Tareas a realizar paso a paso:

- 1. Lanzar una instancia AWS EC2 Ubuntu abriendo los puertos 22, 80 y 443 en el Grupo de Seguridad.
- 2. Diseñar un script `user_data.sh` que instale Docker y Docker Compose automáticamente al iniciar la instancia.
- 3. Crear un fichero `docker-compose.yml` que defina un servicio web frontal (Proxy Inverso) y un contenedor backend en una red interna privada.
- 4. Configurar el proxy para gestionar la terminación SSL en el puerto 443 y redirigir el tráfico de forma transparente hacia el contenedor de la aplicación.
- 5. Comprobar el funcionamiento accediendo a la IP pública o dominio de la instancia EC2 sin necesidad de intervención por SSH.

7.4. Práctica 4: Monitorización de Logs en Tiempo Real (GoAccess) y Prevención de Intrusiones (Fail2ban)

- • Criterios evaluados: RA2.g, RA2.j.
- • Objetivo: Implementar un sistema interactivo de monitorización de tráfico web y un sistema de detección y prevención de intrusiones para bloquear ataques automatizados.

Desarrollo de la Práctica 4: Tareas a realizar paso a paso:

- 1. Instalar y configurar **GoAccess** en el servidor web Apache.
- 2. Configurar un panel de control HTML en tiempo real servido en el directorio `/var/www/html/monitor.html` protegido con autenticación básica.
- 3. Instalar **Fail2ban** y crear un archivo de configuración `/etc/fail2ban/jail.local` para proteger el servidor Apache.
- 4. Simular desde una máquina cliente una ráfaga de peticiones erróneas (fuerza bruta o escaneo de carpetas no existentes 404).
- 5. Comprobar en los registros de Fail2ban (`/var/log/fail2ban.log`) y en `iptables -L -n` que la IP atacante ha sido bloqueada automáticamente.